

Accurate Inference With Inaccurate RRAM Devices: A Joint Algorithm-Design Solution

GOURANGA CHARAN¹ (Graduate Student Member, IEEE),

ABINASH MOHANTY¹ (Member, IEEE),

XIAOCONG DU¹ (Graduate Student Member, IEEE),

GOKUL KRISHNAN¹ (Graduate Student Member, IEEE),

RAJIV V. JOSHI² (Fellow, IEEE), and YU CAO¹ (Fellow, IEEE)

¹School of Electrical, Computer and Energy Engineering, Arizona State University, Tempe, AZ 85287 USA

²IBM Thomas J. Watson Research Center, Yorktown Heights, NY 10598 USA

CORRESPONDING AUTHORS: G. CHARAN (gcharan@asu.edu) and Y. CAO (ycao@asu.edu)

This work was supported in part by the Center for Brain-Inspired Computing (C-BRIC), one of six centers in JUMP, in part by the Semiconductor Research Corporation (SRC) program sponsored by DARPA, and in part by the National Science Foundation (CCF #1715443).

ABSTRACT Resistive random access memory (RRAM) is a promising technology for energy-efficient neuromorphic accelerators. However, when a pretrained deep neural network (DNN) model is programmed to an RRAM array for inference, the model suffers from accuracy degradation due to RRAM nonidealities, such as device variations, quantization error, and stuck-at-faults. Previous solutions involving multiple read-verify-write (R-V-W) to the RRAM cells require cell-by-cell compensation and, thus, an excessive amount of processing time. In this article, we propose a joint algorithm-design solution to mitigate the accuracy degradation. We first leverage knowledge distillation (KD), where the model is trained with the RRAM nonidealities to increase the robustness of the model under device variations. Furthermore, we propose random sparse adaptation (RSA), which integrates a small on-chip memory with the main RRAM array for postmapping adaptation. Only the on-chip memory is updated to recover the inference accuracy. The joint algorithm-design solution achieves the state-of-the-art accuracy of 99.41% for MNIST (LeNet-5) and 91.86% for CIFAR-10 (VGG-16) with up to 5% parameters as overhead while providing a 15–150× speedup compared with R-V-W.

INDEX TERMS Convolution neural networks, device nonidealities, model robustness, neuromorphic computing, random sparse adaptation (RSA), resistive random access memory (RRAM).

I. INTRODUCTION

TODAY deep neural networks (DNNs) have achieved or even surpassed human-level performance in many fields, such as image recognition [1], natural language processing [2], and robotics. DNNs are data-intensive and require extensive mathematical computations. The basic operations of a DNN are a dot product of vector-matrix and matrix-matrix multiplication. GPU [4] is a popular hardware platform for inference due to their highly parallel architecture, which speeds up the computation. However, GPUs, in general, consume high energy and computational resources in both training and inference operations [3], [5].

On the other side, the main driving force of semiconductor design, CMOS technology scaling, is approaching the limit and becomes increasingly difficult to deliver the computation power that is needed for DNNs. In addition, conventional CMOS architecture faces the memory wall, i.e., von-Neumann bottleneck [7], further complicating the

challenges to achieve high-performance and energy-efficient computing. In this context, there is an urgent need for hardware acceleration, exploring beyond-traditional CMOS technology, architectural, and algorithmic solutions [6].

Nonvolatile memory (NVM)-based processing-in-memory (PIM) architectures, such as the crossbar, have demonstrated the potential to speed up the multiply-and-accumulate (MAC) operations in DNNs, achieving high energy efficiency with low latency [8], [9], [11]. Among them, resistive random access memory (RRAM) is a promising technology of choice, with high integration density, fast read speed, long endurance, low access power, and good compatibility with CMOS fabrication [10]. RRAM-based PIM architectures avoid data movement altogether, providing a fast and efficient alternative to conventional CMOS-based design.

However, compared with GPUs that perform computation in high precision, the computation in an RRAM device is limited by finite quantization levels. Furthermore, RRAM

operations in the analog domain [12] is associated with several challenges, such as high device-to-device variation [13], [14], stuck-at-faults (SAFs) [15], and limited ON/OFF ratio. The stochastic process variations in RRAM cause deviation in the programmed conductance value from the pre-trained value, resulting in incorrect weight in the model. The inference accuracy thus drops significantly compared with the software baseline accuracy [22].

Conventional approaches, such as closed-loop-on-device (CLD) [16], trained the RRAM array with gradient descent by iteratively sensing the difference between the actual output and the target output and programming the RRAM array until convergence. Other approaches, such as open-loop-off-device [17], applied a pretrained model to calculate the expected resistance values of the devices, and then, programming and sensing are conducted over a loop, read-verify-write (R-V-W), until the resistance values converge to the expected values. However, R-V-W involves an expensive feedback control and multiple writes to the RRAM. Since RRAM Write is much slower than that in on-chip memory, e.g., SRAM, R-V-W requires a long time to recover the accuracy.

More recent approaches [22] attempted to embed device issues into DNN training and minimize their impact on the accuracy. Recently, Lin *et al.* [18] proposed a simulation framework to compute and model the error rates of the memristor computation in the DNN model to partially recover the inference accuracy. Other work such as [19] adopted a dynamic fixed-point data representation format to minimize the unused most significant bits (MSBs) and propose a variation-aware training (VAT) methodology with noise injection during the training of the DNN model. In VAT [20], [21], [23], RRAM array is read to characterize device variations, and these statistical variations are then embedded to train the neural network. However, these approaches require computationally expensive retraining for each RRAM device. Although previous VAT approaches help to recover the inference accuracy partially, there is still an accuracy drop compared with the software baseline accuracy. To realize the performance advantages of RRAM in PIM and completely recover the accuracy, we propose a novel joint algorithm-design solution with the minimum area overhead. The key contributions of this article are as follows.

- 1) A knowledge distillation (KD)-based approach to transfer a DNN inference model to a realistic RRAM platform, mitigating the accuracy loss.
- 2) An online adaptation solution postmodel mapping that rapidly and completely recovers the accuracy, leveraging a minimum addition of SRAM.
- 3) A joint algorithm-design solution integrating realistic device statistics, algorithms, and design approaches altogether, to minimize the cost and to fully recover the RRAM inference accuracy.

The organization of this article is as follows. In Section II, we evaluate the effects of various RRAM nonidealities on the inference accuracy for MNIST and CIFAR-10 data sets. In Section III, we discuss the proposed algorithm-design solution consisting of a premapping software solution and

a postmapping adaptation method. Section IV presents the experiments on multiple DNNs. In Section V, we demonstrate the efficacy of our proposed solution and further discuss the area overhead.

II. IMPACT OF NONIDEALITIES IN RRAM

Utilizing the natural current accumulation feature, the RRAM-based crossbar performs complex multiplication operations efficiently, without requiring multiple access across the memory hierarchy. The input to the crossbar is the word line (WL) voltage, and the resultant output of the convolution and fully connected layer multiplication operation is the total current read at the bitline (BL). This can be represented as follows:

$$I(j) = \sum_{k=0}^{N-1} G(j, k) \cdot V(j) \quad (1)$$

where $V \in (j = 1, \dots, N - 1)$ is the vector of input voltages fed to the WL, G is the conductivity matrix representing the parameters of the RRAM, and I is the total current per column, i.e., the resultant MAC output of the corresponding neuron. Fig. 1 shows the RRAM-based crossbar architecture with weights mapped as the RRAM conductances.

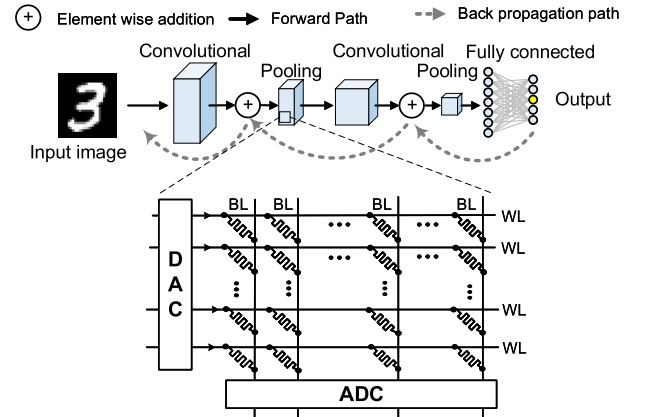


FIGURE 1. Mapping of a DNN network to a realistic RRAM. Each layer of the DNN is mapped on to the array for acceleration.

Despite the advantages of area efficiency, low latency, and static power reduction [8], in using RRAM-based in-memory computing (IMC) accelerators, RRAM suffers from various nonidealities and variations in the analog domain. Nonidealities, such as quantization error, SAFs, and Write variations, need to be accounted for while designing an RRAM-based crossbar architecture. These RRAM nonidealities cause deviation of the DNN parameters from the pretrained model, leading to severe loss in inference accuracy.

A. STUCK-AT-FAULTS

Along with quantization and write variation, one of the other common issues that degrade the performance of RRAM devices is SAFs [15]. Due to the variations in the fabrication process, some of the devices are always at high-resistance state, i.e., SF1 or low-resistance state, i.e., SF0. Previous

works [15] have reported that SF1 and SF0 are assumed to affect 9.04% and 1.75% of the devices. SAFs being a derivative of the fabrication process, it is hard to avoid them.

B. DEVICE QUANTIZATION

In deep learning, a 32-bit floating-point (FP32) format is used for training and inference purposes. Recent works [26] have achieved an 8-bit fixed-point precision of weights and activations without incurring any inference accuracy loss. However, unlike GPUs and CPUs, RRAM devices are limited by finite quantization levels [13]. Hence, an 8-bit quantization of parameters is not a viable solution for RRAM-based crossbars. In this article, we perform a more conservative 4-bit quantization for the model parameters, i.e., weights and biases. We observe inference accuracy degradation for both the MNIST and CIFAR-10 data sets when the weights are quantized to 4 bit, as presented in Fig. 2. The inference accuracy loss is more severe as the depth of the network increases, such as VGG-16, where the accuracy drops from 93.3% when trained in FP32 precision to $\sim 20\%$ in 4-bit (INT4) quantization.

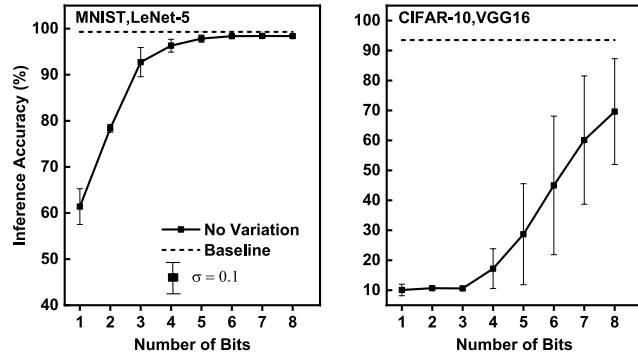


FIGURE 2. Inference accuracy degradation for MNIST (left) and CIFAR-10 (right) data sets due to quantization error. The error bars represent the inference accuracy variations when write variation of $\sigma = 0.1$ is applied to the DNN weights.

C. WRITE VARIATION

Device variations pose a significant challenge to RRAM devices. These variations include temporal, i.e., cycle-to-cycle variations and also spatial, i.e., the device-to-device variations. The spatial variation is due to the fabrication technology and can be improved with precise manufacturing control and advancement in the fabrication process, whereas the temporal variations are an intrinsic property of RRAM resistive switching behavior in the oxide-based memory. Temporal variations are due to the stochastic nature of the formation and rupture of a conductive filament, i.e., oxygen vacancies generation and migration process. Sahay and Suri [24] provided a comprehensive study of the various sources of the variability in RRAM devices. In the simulation, we model the stochastic write variation following a lognormal distribution [19]. This signifies that during on-state, the RRAM resistance will vary, as shown in the following equation:

$$r' \leftarrow e^{\theta} \cdot r \quad (2)$$

where $\theta \sim \mathcal{N}(0, \sigma^2)$, r is the ideal resistance value to be programmed, r' is the actual value programmed, and $\mathcal{N}$ is the normal distribution with 0 mean and σ^2 variance. In reality, the variation in Write depends on both the RRAM conductance values and the input voltage. Based on [19], we model the RRAM Write variation with a constant $\sigma = 0.1$ in this article to reduce the computation cost in large DNNs. We will continue the statistical characterization of RRAM devices and improve the model in future work. In Fig. 3, we show the combined effect of all the nonidealities on inference accuracy for both the MNIST and CIFAR-10 data sets.

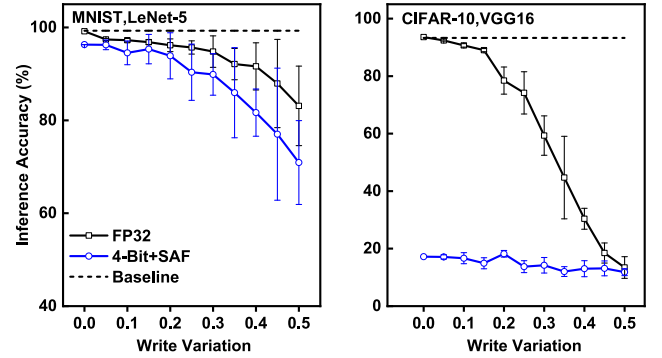


FIGURE 3. Impact of all the nonidealities combined on the inference accuracy for MNIST (left) and CIFAR-10 (right).

Due to these RRAM nonidealities, the pretrained model parameters when programmed to an RRAM device deviate from the ideal values, resulting in significant inference accuracy degradation. The impact is even more severe for deeper networks such as VGG-16, as shown in Figs. 2 and 3. The nominal ON- and OFF-state resistances are set to 10 k Ω –1 M Ω for this article. Table 1 consists of all the nonidealities considered for this study. Previous device-level [16], [17] approaches are time-consuming and do not exploit the inherent redundancy present in modern machine learning algorithms. Similarly, the previous software approaches [19], [20] partly recover the inference accuracy, and there is still a significant drop compared with the baseline model. Therefore, it is essential to seamlessly integrate algorithms and design approaches together to minimize the cost and to fully recover the inference accuracy.

III. JOINT ALGORITHM-DESIGN SOLUTION

RRAM device adds variation to the parameters of the pretrained DNN model, leading to accuracy degradation. For a data set, there exist multiple local minima that reach similar accuracy. Therefore, to mitigate the accuracy loss, we only need to adapt the model to another local minimum and ensure model-level accuracy when mapped to RRAM. Hence, to increase the robustness of the model under RRAM variation, we propose knowledge transfer-based training, where an ideal model (trained in FP32 precision) acts as the teacher model and guides the training of the student model, which is trained with the RRAM nonidealities added, as mentioned in Table 1. In this section, we discuss the proposed method consisting of a model transfer algorithm and a postmapping online adaptation method.

TABLE 1. Assumption of major types of RRAM device nonidealities.

Parameters	Values
Write variation (normal distribution) [19]	$r' \leftarrow r.e^\theta; \theta \sim \mathcal{N}(0, \sigma^2)$
Quantization level	16
Stuck-at-high(SF1) [15]	1.75%
Stuck-at-low(SF0) [15]	9.04%
R_{on}/R_{off} [29]	1000

r is the ideal resistance value to be programmed, r' is the actual value programmed. $\sigma = 0.1$

A. PREMAPPING ALGORITHM: VAT USING KD

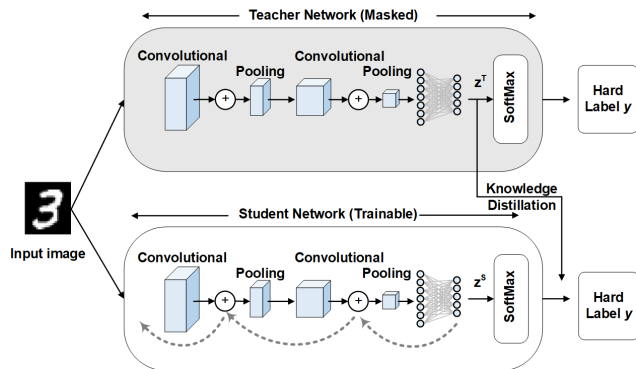
KD [25] facilitates the transfer of information from the teacher model. It helps the student model to mimic the teacher's output as close as possible, thereby resulting in a well-trained student model. The student model is trained by minimizing a loss function in which the target is the distribution of class probabilities as predicted by the teacher model, i.e., the softmax function on the teacher's logits. However, instead of using the vanilla softmax output (hard logits), KD uses the soft logits, which is obtained by incorporating a temperature parameter of τ in the softmax function, as shown in Fig. 4. The softened probability distribution p_i of class i is calculated from logits z as in (3)

$$p_i = \frac{\exp(\frac{z_i}{\tau})}{\sum_j \exp(\frac{z_j}{\tau})} \quad (3)$$

where τ is the softmax temperature. The underlying principle is that as we increase the temperature in the softmax function, the probability distribution provides much more tangible information, i.e., similarity and dissimilarity between the classes.

Let f_S and f_T be the output functions of the student model S and the teacher model T , respectively, and θ_T and θ_S be the trainable parameters of the teacher model and student model, respectively. x_i are the training data samples. KD loss l_{KD} can be expressed as in the following equation:

$$l_{KD} = \sum_{x_i \in \mathcal{X}} L \left(\text{softmax} \left(\frac{f_T(x_i, \theta_T)}{\tau} \right), \text{softmax} \left(\frac{f_S(x_i, \theta_S)}{\tau} \right) \right) \quad (4)$$


FIGURE 4. KD-based teacher-student training. KD uses the soft output of the teacher as a target for the student along with the hard labels for training.

where L is the loss function minimizing the difference between the student and teacher model's softmax output.

The primary focus of this article is to mitigate the inference accuracy degradation due to RRAM device nonidealities. Therefore, we explore model transfer-based VAT methodology as a technique to increase model robustness and mitigate accuracy loss. First, for every training batch during training, RRAM nonidealities are added to the parameters of the student model, i.e., we quantize the parameters to 4 bits, add SAFs [15], i.e., set 1.75% of the parameters to 1, and 9.04% of the parameters to 0, and for each convolutional kernel, we generate a random Gaussian noise matrix with zero mean and $\sigma = 0.1$ deviation with the same size of that of the kernel. Similarly, random noise is generated for the fully connected layers. Furthermore, we perform elementwise multiplication to add this generated noise to the DNN parameters.

Second, instead of training the standalone student model with nonidealities, we leverage the converged teacher model's knowledge to facilitate the training of the student model. To further compensate for the inference accuracy loss, we propose random sparse adaptation (RSA), an online sparse adaptation technique that utilizes a small on-chip memory for postmapping adaptation.

TABLE 2. Timing parameters and size comparison of RRAM and on-chip memory, i.e., SRAM.

	RRAM				SRAM
Material	HfO_x	TiO_x	TaO_x	AlO_x	Si
Levels	16	128	4	2	32-bit
Write time	$6.5\ \mu s$	$60\ \mu s$	1.5 ms	4 ms	1 ns
Read time	24-404 ns				1 ns
Array Size	1000x1000				100 cells
Area	$10^6 \times 4F^2$				$10^4 \times 1.5F^2$
F is the feature size of lithography. [12], [13], [28]					

B. POSTMAPPING ADAPTATION: RSA

The conventional way of recovering the accuracy loss in realistic RRAM devices is by using the R-V-W technique, involving multiple reads and write operations to the RRAM device [16], [17]. The time required for recovering the accuracy in R-V-W operation shoots up as the percentage of parameters needed to be retrained increases.

Moreover, compared with device-level RRAM R-V-W adaption, machine learning algorithms can very quickly adapt to changes in parameter space during training. In Table 2, we compare various properties of the standard RRAM with that of an SRAM. In general, the RRAM writes are slow compared with the on-chip memory, i.e., SRAM. Hence, the SRAM with high write speed and reliability is the right technology of choice for postmapping adaptation. The core idea behind RSA is to circumvent this slow RRAM R-V-W operations by using a small, faster on-chip memory-based adaptation to recover the inference accuracy drop due to RRAM nonidealities.

In RSA, a certain percentage of the RRAM cells are first selected and then mapped to the on-chip memory for further

training. The cells are randomly selected so that the entire feature space of the network is covered. The connection between the RRAM array and the on-chip memory is proposed to be hard-wired. Since the cell positions are random and such randomness is independent of the algorithm or the training step, we can hard-wire the random connection between RSA and RRAM input/output. Thus, during inference, the input is passed through both the RRAM and the on-chip memory. The output from both these paths is summed up to generate the final output of the network. Now, to recover the inference accuracy, we propose to train the on-chip memory as part of the postmapping adaptation. The RRAM cells are masked and are set nontrainable. The gradient update, i.e., backpropagation, is only applied to the faster on-chip memory to adapt the overall network. Although there are several benefits of bypassing the RRAM R-V-W operation, the accuracy gain using RSA is associated with an increase in area cost. We analyze the cost involved and measures taken to minimize this overhead in Section V.

C. JOINT ALGORITHM-DESIGN SOLUTION FOR ACCURATE INFERENCE WITH INACCURATE RRAM

In Fig. 5, we show the overall architecture integrating both the software and the hardware solution. The joint algorithm-design technique to mitigate the inference accuracy drop in realistic RRAMs consists of a two-part solution.

- 1) A premapping software solution where we strive toward achieving a more robust and reliable network using KD-based VAT.
- 2) A faster and efficient postmapping hardware solution, where we use a small on-chip memory for online adaptation to compensate for the accuracy degradation, after the KD-based pretrained network is programmed on to the RRAM array. The integration of on-chip memory and RRAM provides flexibility and higher accuracy compared with RRAM only solution.

IV. EXPERIMENTAL SETUP

We present the experimental setup and results in this section, to empirically evaluate our proposed approach two data sets, i.e., CIFAR-10 and MNIST for VGG-16 and LeNet-5 networks, respectively.

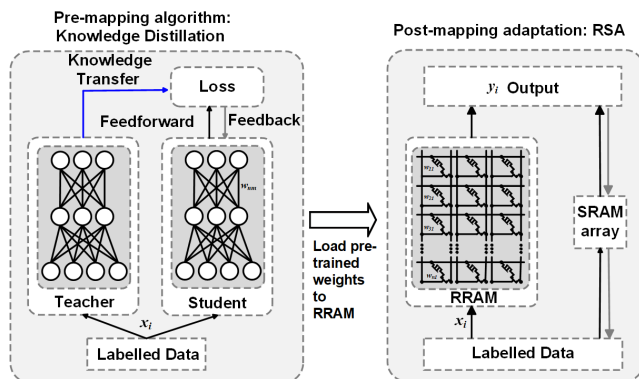


FIGURE 5. Joint algorithm design solution consisting of premapping KD-based training and postmapping on-chip memory-based adaptation.

A. NETWORK STRUCTURES

LeNet-5 consists of two convolutional layers, followed by a max-pooling layer, and two fully connected layers. Rectified linear unit (ReLU) is used as the activation function for all the layers except the final fully connected layer, which has a softmax classifier. The VGG-16 network has 13 convolution layers with a ReLU activation function and batch normalization followed by max-pooling. VGG-16 also has two fully connected layers, where the final output layer has softmax activation.

B. HYPERPARAMETERS AND FRAMEWORK

For CIFAR-10, the learning rate is set to 0.1, and the rate is reduced by a factor of 10 after every 60 epochs. We run the experiments for 160 epochs to make sure that the network reaches a stable solution. For MNIST, we use a learning rate of 0.001 and train for 50 epochs. The KD temperature τ is set to 2 for all the experiments. Cross entropy with softmax function, along with the distillation loss, is used as the loss function for the experiments. We use stochastic gradient descent (SGD) with momentum to train the network in all the experiments. We have used the Tensorflow framework on an NVIDIA GeForce GTX 1080 Ti platform for all the experiments.

V. RESULTS

Here, we describe the notations used in the figures to be consistent and specific in our results. The baseline/teacher-FP refers to the full-sized software model trained in full precision (FP32), and the inference accuracy reported for the baseline model is without incorporating any nonidealities or variations. The teacher-realistic and the student model refer to a similar-sized network, such as the teacher-FP model. In both cases, the network is trained with the RRAM nonidealities incorporated within the model itself. The only difference is that the student model is trained with the teacher model's knowledge using KD, whereas the teacher-realistic model is trained standalone. The KD+RSA accuracy corresponds to the scenario where both KD-based VAT and RSA are performed.

A. PREMAPPING ALGORITHM RESULTS (KD)

In this section, we explore the efficacy of our proposed KD-based VAT methodology using LeNet-5 for the MNIST data set and VGG-16 for the CIFAR-10 data set. Fig. 6 shows the inference accuracies for our teacher-FP, the teacher-realistic, and the student model. For both the experiments, we have used a write variation with $\sigma = 0.1$ for training and testing purposes. For MNIST, the inference accuracy for the teacher-FP model is 99.33%. The teacher-realistic and student model achieve 95.15% and 97.43% inference accuracy, respectively. Similarly, for CIFAR-10, the software baseline accuracy for the teacher-FP model is 93.35%, and for the teacher-realistic and student model, the inference accuracies are 82.6% and 87.13%, respectively.

We observe that knowledge transfer-based training performs consistently better than standalone VAT, achieving a 2.28% and 4.53% improvement in inference accuracy for

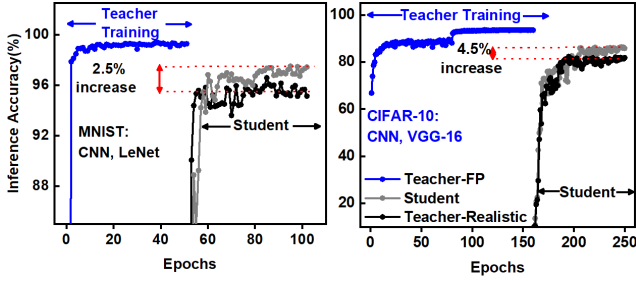


FIGURE 6. Inference accuracy comparison of the baseline teacher model with the student-only and student with KD model for MNIST (left) and CIFAR-10 (right) data sets.

both MNIST and CIFAR-10 data sets, respectively. Both the results support our argument that an efficient pretraining algorithm is an absolute necessity to obtain a robust model, capable of reducing the inference accuracy degradation using inaccurate RRAMs.

B. POSTMAPPING ADAPTATION RESULTS

Accurate write operations to RRAM are slower compared with that of SRAM and limit the performance of the R-V-W operation. Previous experimental results have shown that RRAM write operations usually take $6.5 \mu\text{s}$ – 4.5 ms , whereas SRAM write takes $\sim 1 \text{ ns}$ [13]. More recently, SiO_x -based RRAM devices [30] have shown promising results with write operation in nanoseconds. Considering in most previous works, the Write time is in μs – ms , we choose this range in our current simulation and will keep updating the experiments based on technology progress. We estimate the performance of R-V-W and RSA using the timing parameters listed in Table 2. Table 3 shows the percentage of RRAM cells required to R-V-W and the time taken to achieve the inference accuracy level for two representative data sets, MNIST and CIFAR-10. In this case, the parameters are sorted first, and the top $x\%$ percentage of parameters is used for R-V-W, where x is a user-defined hyperparameter. Hence, for deeper networks such as VGG-16, 100% of the top-ranked parameters are required for R-V-W, and even then, the device suffers from accuracy degradation of 4.87%. Since in RSA, the RRAM cells are masked from write operations, and only the SRAM is updated during each training, the adverse impact of slow RRAM write is fully averted, which promises the speedup in accuracy recovery. In Fig. 7, we compare the accuracy improvement and the time required to achieve the accuracy for R-V-W and RSA using CIFAR-10 on VGG-16. Here, for RSA alone, we need 10% of the network parameters to be mapped on to the on-chip memory. RSA alone achieves a $10\times$ improvement in the time required to reach the baseline accuracy.

Although standalone RSA is capable of recovering the accuracy drop, the additional overhead of 10% of parameters in the on-chip memory becomes a major concern. For example, VGG-16 has 15.3 million parameters. The need for 10% of the parameters would mean 1.53 million parameters to be programmed to the on-chip memory. This will become a huge bottleneck as the network size increases and would not only increase the overall area of the chip, but with an increase in

TABLE 3. R-V-W accuracy and computation time for the MNIST and CIFAR-10 data sets.

% of RRAM cells for R-V-W		10%	50%	70%	90%	100%
MNIST	Accuracy (%)	85.49	98.79	99.12	99.18	99.24
	Time (s)	20.56	102	143	184	203
CIFAR-10	Accuracy (%)	22.33	67.27	78.41	85.17	88.43
	Time (s)	2295	11475	16065	20655	22720

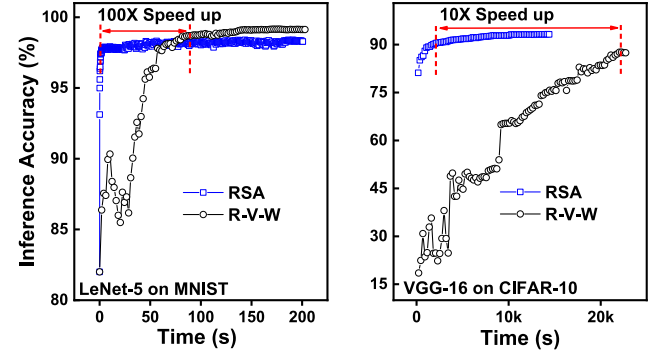


FIGURE 7. Comparing the inference accuracy and the speedup of R-V-W and RSA for MNIST (left) and CIFAR-10 (right). RSA rapidly recovers the accuracy but with an area cost.

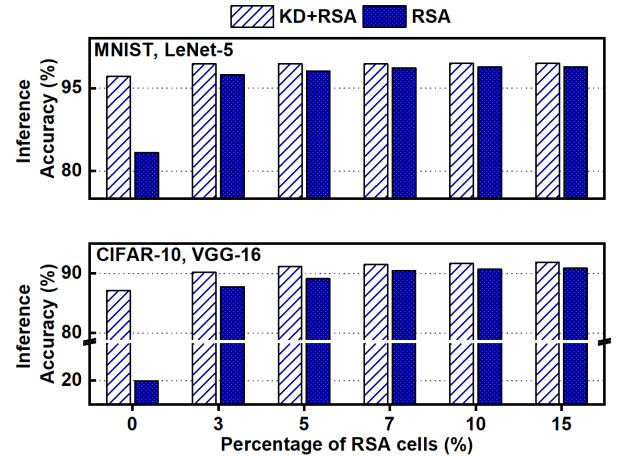


FIGURE 8. Comparison of the inference accuracy and percentage of network parameters that need to be programmed to RSA for MNIST (top) and CIFAR-10 (bottom). KD+RSA together reduces the parameter requirement significantly to 3% from 7% for MNIST and to 5% from 10% for CIFAR-10.

parameters for online adaptation, the overall power requirement of the chip will shoot up exponentially.

The goal is to reduce the overall area overhead required to recover the inference accuracy. The joint algorithm-design solution helps in reducing the total percentage of parameters required for online adaptation. In Fig. 8, we compute the percentage of parameters required for RSA when we have a KD-based variation-aware pretrained network programmed on to the RRAM for both CIFAR-10 and MNIST. Hence, the percentage of parameters required for RSA drops down to 5% from 10% for CIFAR-10 and to 3% from 7% for MNIST.

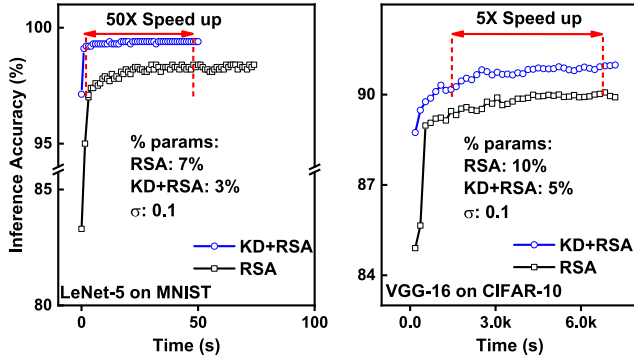


FIGURE 9. Comparison of the speedup of RSA with KD+RSA alone for MNIST (left) and CIFAR-10 (right).

TABLE 4. Inference accuracy comparison of the proposed method with other existing works.

Dataset	Network	Approach	No. of Levels	Accuracy (%)	
				Baseline	RRAM
MNIST	LeNet-5	R-V-W	16	99.51	99.24
		KD	16		97.43
		KD+RSA	16		99.13
CIFAR-10	VGG-16	2-Layer MLP	64	85.00	79.80
		R-V-W	16	93.35	88.43
		KD	16		87.13
		KD+RSA	16		91.86
		Others [19]	256		80.10

For example, now, we just need 765k parameters compared with 1.53 million parameters in the RSA-only scenario. This is a significant reduction in the total parameter requirement. Hence, the premapping algorithm successfully mitigates one of the limitations of the RSA algorithm.

In Fig. 9, we compare the inference accuracy and the time required for KD+RSA with RSA alone on CIFAR-10 with VGG-16. KD+RSA, with up to 5% of the parameters on the on-chip memory, outperforms all the previous approaches and achieves the state-of-the-art inference accuracy of 91.86% and 99.13% for CIFAR-10 and MNIST, respectively. Furthermore, KD+RSA achieves a 10–150 $\times$ acceleration compared with RSA alone and R-V-W. Table 4 presents the inference accuracies for our KD+RSA method and compares its performance with the existing approaches. This further supports our argument toward the need for a joint algorithm-design solution to mitigate the inference accuracy degradation using inaccurate RRAM.

VI. CONCLUSION AND DISCUSSION

Although the postmapping algorithm, such as RSA and KD+RSA, helps to recover the accuracy drop from using inaccurate RRAMs, it comes with an extra hardware cost, i.e., the area overhead. To evaluate the hardware efficiency of the proposed method, we use NeuroSim [27], an architectural analysis tool for evaluating the area cost of the main model and the additional on-chip memory. In our proposed methodology, we use 4 bit for the RRAM and 32 bit for

the on-chip memory. We map the RRAM software model on the crossbar-based hardware structure, and for the on-chip memory, we use an SRAM-based array in NeuroSim [27]. We perform a complete area evaluation, which includes all the components in the tiled architecture of the chip for the main RRAM array and the RSA cells. The architecture consists of a hierarchical computing structure with 16 processing elements (PEs) per tile. A point-to-point interconnect is chosen for the routing within the main chip and RSA cells on the SRAM array. Compared with the dense RRAM device, the footprint of the SRAM cell is much higher. Therefore, in Fig. 10, we observe that for CIFAR-10, with 15% SRAM parameters leads to a 26% increase in the total area cost. In Table 5, we show the exact area required for the subarray and the peripherals. SRAM has a relatively larger portion of the peripheral area. This is primarily because of the addition of peripheral circuits, such as write-driver, BL conditioning circuits, required by the SRAM array.

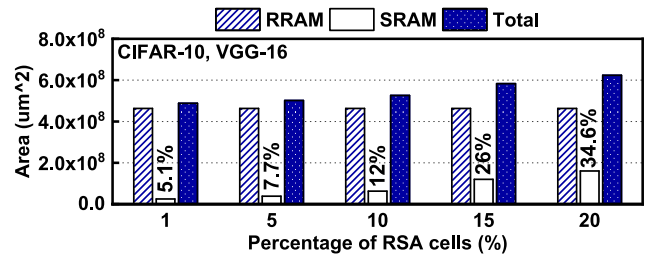


FIGURE 10. Area breakdown for both RRAM and on-chip SRAM memory for the proposed methodology.

Since KD increases the robustness of the model, it helps to reduce the percentage of parameters required for online adaptation from 10% to 5%, for CIFAR-10, thereby reducing the area overhead from 12% to 7.7%. In addition, we perform the energy analysis for the proposed hybrid solution for CIFAR-10 with the VGG-16 network. The total energy consumption of the main RRAM array and the peripherals is 1.23 mJ, which includes the dynamic energy of 0.16 mJ and the leakage energy of 1.07 mJ. The total energy consumption of the SRAM array (RSA) for 5% and 10% parameters is 0.397 and 0.8 mJ, respectively. Therefore, the total energy overhead for KD+RSA and RSA alone is 32.27% and 65.04%, respectively, compared with the main RRAM array. Overall, the joint algorithm-design solution not only achieves the baseline accuracy faster but also minimizes the area overhead with additional energy consumption.

In summary, IMC using NVMs, such as RRAM, has the potential to solve the bandwidth bottleneck issues in today's systems and to be the building blocks of fast, power-efficient deep learning accelerators. However, RRAM suffers from accuracy degradation due to various nonidealities, such as quantization error, write variations, and SAFs. In order to efficiently mitigate the effects of these nonidealities in inaccurate RRAM, we need to optimize both the software training phase and the posttraining phase, i.e., hardware mapping. This article proposes a joint algorithm-design solution: first, during training, the teacher model's knowledge is effectively

TABLE 5. Area breakup results for RRAM and SRAM using NeuroSim.

Model	% of Params	Total Area (μm^2)	Total Subarray Area (μm^2)	Peripheries (μm^2)
VGG-16 (RRAM)	100	3.80E+08	2.12E+08	1.21E+08
VGG-16 (SRAM)	10	6.84E+07	2.65E+07	2.85E+07
	5	4.18E+07	1.62E+07	1.76E+07
	1	2.72E+07	1.04E+07	1.15E+07

transferred, while training a student model with the nonidealities incorporated; then, perform a postmapping adaptation, where we randomly select a portion of the network parameters and map them on to the on-chip memory for further training. Although we do not explicitly consider sneak paths and other noise of the peripheral circuits, during adaptation, the final loss function is computed based on the output from the SRAM and the main memory, which includes those nonidealities. Hence, the trained hybrid model can mitigate their impact. Moreover, RRAM suffers from cell disturbances and long-term retention degradation during continuous inference operations. The inference accuracy degradation due to these nonidealities can be mitigated by performing the postmapping adaptation step (RSA). The proposed methodology achieves the state-of-the-art inference accuracy of 99.13% and 91.86% for MNIST and CIFAR-10 data sets with up to 5% area overhead, respectively. Thus, it is essential to integrate device statistics, algorithms, and design approaches all together to minimize the cost and fully recover the accuracy while implementing neuromorphic systems using nonideal RRAMs.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017.
- [2] A. Graves, A. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, May 2013, pp. 6645–6649.
- [3] X. Du, Z. Li, Y. Ma, and Y. Cao, "Efficient network construction through structural plasticity," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 3, pp. 453–464, Sep. 2019.
- [4] H. Cui, H. Zhang, G. R. Ganger, P. B. Gibbons, and E. P. Xing, "GeePS: Scalable deep learning on distributed GPUs with a GPU-specialized parameter server," in *Proc. 11th Eur. Conf. Comput. Syst. (EuroSys)*, no. 4, New York, NY, USA: ACM, 2016, pp. 4.1–4.16, doi: [10.1145/2901318.2901323](https://doi.org/10.1145/2901318.2901323).
- [5] S. B. Eryilmaz, D. Kuzum, S. Yu, and H.-S.-P. Wong, "Device and system level design considerations for analog-non-volatile-memory based neuromorphic architectures," in *IEDM Tech. Dig.*, Dec. 2015, pp. 4.1.1–4.1.4, doi: [10.1109/IEDM.2015.7409622](https://doi.org/10.1109/IEDM.2015.7409622).
- [6] X. Du, G. Krishnan, A. Mohanty, Z. Li, G. Charan, and Y. Cao, "Towards efficient neural networks on-a-chip: Joint hardware-algorithm approaches," in *Proc. China Semiconductor Technol. Int. Conf. (CSTIC)*, Mar. 2019, pp. 1–5.
- [7] S. A. McKee, "Reflections on the memory wall," in *Proc. 1st Conf. Comput. Frontiers Comput. Frontiers (CF)*, 2004, p. 162, doi: [10.1145/977091.977115](https://doi.org/10.1145/977091.977115).
- [8] A. Shafiee et al., "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2016, pp. 1–12.
- [9] G. W. Burr et al., "Neuromorphic computing using non-volatile memory," *Adv. Phys. X*, vol. 2, no. 1, pp. 89–124, Nov. 2016, doi: [10.1080/23746149.2016.1259585](https://doi.org/10.1080/23746149.2016.1259585).
- [10] C.-C. Chang et al., "Challenges and opportunities toward online training acceleration using RRAM-based hardware neural network," in *IEDM Tech. Dig.*, Dec. 2017, pp. 11.6.1–11.6.4, doi: [10.1109/IEDM.2017.8268373](https://doi.org/10.1109/IEDM.2017.8268373).
- [11] L. Ni, Z. Liu, H. Yu, and R. V. Joshi, "An energy-efficient digital ReRAM-crossbar-based CNN with bitwise parallelism," *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 3, pp. 37–46, 2017.
- [12] S. R. Lee et al., "Multi-level switching of triple-layered TaOx RRAM with excellent reliability for storage class memory," in *Proc. Symp. VLSI Technol. (VLSIT)*, Jun. 2012, pp. 71–72, doi: [10.1109/VLSIT.2012.6242466](https://doi.org/10.1109/VLSIT.2012.6242466).
- [13] F. Alibart, L. Gao, B. D. Hoskins, and D. B. Strukov, "High precision tuning of state for memristive devices by adaptable variation-tolerant algorithm," *Nanotechnology*, vol. 23, no. 7, Feb. 2012, Art. no. 075201, doi: [10.1088/0957-4484/23/7/075201](https://doi.org/10.1088/0957-4484/23/7/075201).
- [14] K. C. Hsu et al., "A study of array resistance distribution and a novel operation algorithm for WOx ReRAM memory," in *Proc. Extended Abstr. Int. Conf. Solid State Devices Mater.*, Sep. 2015, pp. 1168–1169.
- [15] C.-Y. Chen et al., "RRAM defect modeling and failure analysis based on march test and a novel squeeze-search scheme," *IEEE Trans. Comput.*, vol. 64, no. 1, pp. 180–190, Jan. 2015, doi: [10.1109/TC.2014.12](https://doi.org/10.1109/TC.2014.12).
- [16] M. Hu, H. Li, Y. Chen, Q. Wu, and G. S. Rose, "BSB training scheme implementation on memristor-based circuit," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl. (CISDA)*, Apr. 2013, pp. 80–87, doi: [10.1109/CISDA.2013.6595431](https://doi.org/10.1109/CISDA.2013.6595431).
- [17] B. Liu et al., "Reduction and IR-drop compensations techniques for reliable neuromorphic computing systems," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2014, pp. 63–70, doi: [10.1109/ICCAD.2014.7001330](https://doi.org/10.1109/ICCAD.2014.7001330).
- [18] M.-Y. Lin et al., "DL-RSIM: A simulation framework to enable reliable ReRAM-based accelerators for deep learning," in *Proc. Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2018, pp. 1–8.
- [19] Y. Long, X. She, and S. Mukhopadhyay, "Design of reliable DNN accelerator with un-reliable ReRAM," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 1769–1774, doi: [10.23919/DATE.2019.8715178](https://doi.org/10.23919/DATE.2019.8715178).
- [20] B. Liu, H. Li, Y. Chen, X. Li, Q. Wu, and T. Huang, "Vortex: Variation-aware training for memristor crossbar," in *Proc. 52nd Annu. Design Autom. Conf. (DAC)*, 2015, pp. 1–6, doi: [10.1145/2744769.2744930](https://doi.org/10.1145/2744769.2744930).
- [21] L. Chen et al., "Accelerator-friendly neural-network training: Learning variations and defects in RRAM crossbar," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2017, pp. 19–24, doi: [10.23919/DATE.2017.7926952](https://doi.org/10.23919/DATE.2017.7926952).
- [22] I. Chakraborty, D. Roy, and K. Roy, "Technology aware training in memristive neuromorphic systems for nonideal synaptic crossbars," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, no. 5, pp. 335–344, Oct. 2018.
- [23] M. Klachko, M. R. Mahmoodi, and D. Strukov, "Improving noise tolerance of mixed-signal neural networks," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2019, pp. 1–8.
- [24] S. Sahay and M. Suri, "Recent trends in hardware security exploiting hybrid CMOS-resistive memory circuits," *Semicond. Sci. Technol.*, vol. 32, no. 12, Dec. 2017, Art. no. 123001.
- [25] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*. [Online]. Available: <https://arxiv.org/abs/1503.02531>
- [26] ArXiv Formatted R. Banner, I. Hubara, E. Hoffer, and D. Soudry, "Scalable methods for 8-bit training of neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2018, pp. 5145–5153.
- [27] X. Peng, S. Huang, Y. Luo, X. Sun, and S. Yu, "DNN+NeuroSim: An end-to-end benchmarking framework for Compute-in-Memory accelerators with versatile device technologies," in *IEDM Tech. Dig.*, Dec. 2019, pp. 32.5.1–32.5.4.
- [28] L. Gao, P.-Y. Chen, and S. Yu, "Programming protocol optimization for analog weight tuning in resistive memories," *IEEE Electron Device Lett.*, vol. 36, no. 11, pp. 1157–1159, Nov. 2015, doi: [10.1109/LED.2015.2481819](https://doi.org/10.1109/LED.2015.2481819).
- [29] H. Li et al., "Variation-aware, reliability-emphasized design and optimization of RRAM using SPICE model," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2015, pp. 1425–1430.
- [30] L. Montesi et al., "Nanosecond analog programming of substoichiometric silicon oxide resistive RAM," *IEEE Trans. Nanotechnol.*, vol. 15, no. 3, pp. 428–434, May 2016.



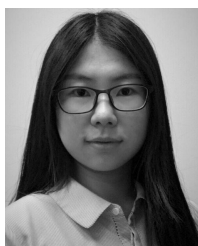
GOURANGA CHARAN (Graduate Student Member, IEEE) received the B.Tech. degree in instrumentation engineering from the Indian Institute of Technology Kharagpur, India, in 2015. He is currently pursuing the Ph.D. degree in electrical engineering with the Arizona State University, Tempe, AZ, USA.

He was an IC Design Engineer at Broadcom Inc., Bengaluru, India, for two years. His current research interests include in-memory computing, robust on-chip learning, joint algorithm-architecture design for learning on-a-chip, and continual learning.



ABINASH MOHANTY (Member, IEEE) received the B.Tech. degree in electronics and communications engineering from the National Institute of Technology, Rourkela, India, in 2010, and the Ph.D. degree in electrical engineering from Arizona State University, Tempe, AZ, USA, in 2018.

He was a Senior Software Engineer in embedded systems with the Samsung Research Institute, New Delhi, India, for three years. He was an Intern with Intel Corporation, Santa Clara, CA, USA, in 2016. His research interests include machine learning and computer vision, computer architecture, hardware-software co-optimization, and device-level reliability modeling.



XIAOCONG DU (Graduate Student Member, IEEE) received the B.S. degree in control engineering from Shandong University, Jinan, China, in 2014, and the M.S. degree in electrical and computer engineering from the University of Pittsburgh, Pittsburgh, PA, USA, in 2016. She is currently pursuing the Ph.D. degree in electrical engineering with Arizona State University, Tempe, AZ, USA.

Her research focuses on efficient machine learning algorithm design, including model compression, continual-learning, neural-architecture search, and brain-inspired computing.



GOKUL KRISHNAN (Graduate Student Member, IEEE) received the B.Tech. degree in electronics and communication engineering from the Government Model Engineering College, Kochi, India, in 2016. He is currently pursuing the Ph.D. degree in electrical engineering with Arizona State University, Tempe, AZ, USA.

His current research interests include neuromorphic hardware design for deep learning, joint algorithm-architecture design for learning on-a-chip, model compression of DNNs, and performance modelling for CMOS and post-CMOS-based hardware architectures.



RAJIV V. JOSHI (Fellow, IEEE) received the B.Tech. degree from IIT Bombay, Bombay, India, the M.S. degree from MIT, and the Dr.Eng.Sc. degree from Columbia University.

He is currently a Research Staff Member and also a Key Technical Lead at the T. J. Watson Research Center, IBM. His novel interconnects processes and structures for aluminum, tungsten, and copper technologies which are widely used in IBM for various technologies from sub-0.5 μm to 14nm. He has led successfully predictive failure analytic techniques for yield

prediction and also the technology-driven SRAM at the IBM Server Group. He has extensively worked on novel memory designs. He commercialized these techniques. He has received three Outstanding Technical Achievement (OTAs), three highest Corporate Patent Portfolio awards for licensing contributions, holds 60 invention plateaus, and has over 235 US patents and over 354, including international patents. His research interests are in in-memory computation, CNN, DNN accelerators, and quantum computing. He has authored and coauthored over 200 papers. He has given over 45 invited/keynote talks and given several seminars.

Dr. Joshi is a member of the IBM Academy of Technology and a master inventor. He has served as a Distinguished Lecturer for the IEEE CAS and EDS Society. He is currently a Distinguished Lecturer for CEDA. He is a fellow of the ISQED and World Technology Network and distinguished alumnus of IIT Bombay. He serves in the Board of Governors for the IEEE CAS as the industrial liaison. He will and has served on committees of DAC 2019, AICAS 2019, ISCAS, International Symposium on Low Power Electronics and Design (ISLPED), IEEE VLSI design, IEEE CICC, the IEEE International SOI Conference, ISQED, and Advanced Metallization Program Committees. He initiated IBM CAS EDS Symposium at IBM in 2017 and will continue into 2018 with Artificial Intelligence as the focal area. He has served as the General Chair for the IEEE ISLPED. He is an industry liaison for universities as a part of the Semiconductor Research Corporation. Also, he is in the Industry Liaison Committee for the IEEE CAS Society. He has been awarded the prestigious IEEE Daniel Noble Award for 2018. He received the Best Editor Award from the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) journal. He is a recipient of the 2015 BMM Award. He is inducted into New Jersey Inventor Hall of Fame in August 2014, along with pioneer Nicola Tesla. He is a recipient of the 2013 IEEE CAS Industrial Pioneer Award and 2013 Mehboob Khan Award from the Semiconductor Research Corporation. He serves as an Associate Editor for the IEEE TRANSACTIONS ON VLSI SYSTEMS (TVLSI).



YU CAO (Fellow, IEEE) received the B.S. degree in physics from Peking University in 1996, and the M.A. degree in biophysics and the Ph.D. degree in electrical engineering from the University of California, Berkeley, CA, USA, in 1999 and 2002, respectively.

He worked as a Summer Intern at Hewlett-Packard Labs, Palo Alto, CA, USA, in 2000, and at IBM Microelectronics Division, East Fishkill, NY, USA, in 2001. After working as a Post-Doctoral Researcher at the Berkeley Wireless Research Center (BWRC), he is currently a Professor of electrical engineering with Arizona State University, Tempe, AZ, USA. He has published numerous articles and two books on nano-CMOS modeling and physical design. His research interests include neural-inspired computing, hardware design for on-chip learning, and reliable integration of nanoelectronics.

Dr. Cao was a recipient of the 2012 Best Paper Award at the IEEE Computer Society Annual Symposium on VLSI, 2010, 2012, 2013, 2015, and 2016 Top 5% Teaching Award, Schools of Engineering, Arizona State University, the 2009 ACM SIGDA Outstanding New Faculty Award, the 2009 Promotion and Tenure Faculty Exemplar, Arizona State University, the 2009 Distinguished Lecturer of IEEE Circuits and Systems Society, the 2008 Chunhui Award for Outstanding Oversea Chinese Scholars, the 2007 Best Paper Award at the International Symposium on Low Power Electronics and Design, the 2006 NSF CAREER Award, the 2006 and 2007 IBM Faculty Award, the 2004 Best Paper Award at International Symposium on Quality Electronic Design, and the 2000 Beatrice Winner Award at the International Solid-State Circuits Conference. He has served as an Associate Editor for the IEEE TRANSACTIONS ON CAD, and has served on the technical program committee for many conferences.